



**CS5500 Project – ATM System SPECIFICATION & DESIGN**

**MSB**

**ATM**

**Team Members:**

**Dylan Weaver**

**De-Graft Quansah**

**Course:** CS5500-060 – Software Engineering

**Instructor:** Dr. Cyril S. Ku

**Date:** December 08, 2025



# TABLE OF CONTENTS

## Title Page — Page 1

|                  |   |
|------------------|---|
| Title Page ..... | 1 |
|------------------|---|

## Introduction & System Overview

|                                     |     |
|-------------------------------------|-----|
| Introduction .....                  | 6   |
| System Overview .....               | 6–7 |
| Project Objectives .....            | 7   |
| Purpose of the Final Document ..... | 7   |

## Team Information & Project Experience

|                                         |     |
|-----------------------------------------|-----|
| Team Overview and Lessons Learned ..... | 8–9 |
|-----------------------------------------|-----|

---

## Requirements & Specification

### ATM Functional Requirements

|                                           |       |
|-------------------------------------------|-------|
| Logon / Authentication Requirements ..... | 10    |
| Deposit Requirements .....                | 10–11 |
| Withdrawal Requirements .....             | 10    |
| Transfer Requirements .....               | 11    |
| Balance Inquiry Requirements .....        | 11    |
| Receipt Requirements .....                | 11    |

### Use Case Diagram

|                                           |    |
|-------------------------------------------|----|
| ATM System Use Case Diagram (Image) ..... | 12 |
|-------------------------------------------|----|

---

## **Design: Class Diagrams & Class Descriptions**

### **Class Descriptions**

|                                             |    |
|---------------------------------------------|----|
| Account Class Description .....             | 13 |
| CDAccount Description .....                 | 14 |
| CentralDatabase Description .....           | 15 |
| CheckingAccount & ConsumerLoanAccount ..... | 16 |
| Customer Class Description .....            | 17 |
| ErrorHandling Class Description .....       | 18 |
| MSB_ATM Class Description .....             | 19 |
| Screen Class Description .....              | 20 |
| Transaction Class Description .....         | 21 |

### **Static Relationship Diagram**

|                                               |    |
|-----------------------------------------------|----|
| ATM Static Relationship Diagram (Image) ..... | 22 |
|-----------------------------------------------|----|

---

## **Use Case Specifications & Communication Diagrams**

### **Logon Use Case**

|                             |       |
|-----------------------------|-------|
| Use Case Description .....  | 23–24 |
| Communication Diagram ..... | 25    |
| Operation Sequence .....    | 26    |
| Scenario Sequences .....    | 26    |

### **Make Withdrawal**

|                             |       |
|-----------------------------|-------|
| Use Case Description .....  | 27–29 |
| Communication Diagram ..... | 30–31 |
| Operation Sequence .....    | 31    |
| Scenario Sequences .....    | 32    |

**Make Deposit**

|                             |       |
|-----------------------------|-------|
| Use Case Description .....  | 33–34 |
| Communication Diagram ..... | 35    |
| Operation Sequence .....    | 36    |
| Scenario Sequences .....    | 37    |

**Transfer Funds**

|                             |       |
|-----------------------------|-------|
| Use Case Description .....  | 38–40 |
| Communication Diagram ..... | 41    |
| Operation Sequence .....    | 42–43 |
| Scenario Sequences .....    | 43    |

**Check Balance**

|                             |       |
|-----------------------------|-------|
| Use Case Description .....  | 44–45 |
| Communication Diagram ..... | 46    |
| Operation Sequence .....    | 47    |
| Scenario Sequences .....    | 47    |

**Print Receipt**

|                             |       |
|-----------------------------|-------|
| Use Case Description .....  | 48–49 |
| Communication Diagram ..... | 50    |
| Operation Sequence .....    | 51    |
| Scenario Sequences .....    | 51    |

---

**Implementation**

## Full ATM C++ Implementation

|                                                                         |       |
|-------------------------------------------------------------------------|-------|
| ATM Core Classes (Screen, MSB_ATM, Error Handling, CentralDatabase) ... | 52–60 |
| Customer & Account Hierarchy .....                                      | 52–60 |
| Transaction Class Implementation .....                                  | 60–63 |
| Receipt Class Implementation .....                                      | 63    |

---

**Glossary**

Glossary ..... 64–71

**Index**

Index ..... 72–77

---

**Author: Dylan**

**Last Updated 11/6/2025**

## **Introduction**

The MSB ATM System Project is a multi step software engineering project in which our team is a professional consulting and development firm that was hired to design and build an Automated Teller Machine system for Montana Savings Bank (MSB). MSB is a small financial institution with three local branches in Montana that currently relies heavily on tellers, cashiers, and branch managers for daily banking operations. To streamline operations, reduce labor costs, minimize manual errors, and provide 24-hour service to its customers, MSB decided to introduce a modern ATM infrastructure across its branches and nearby public locations.

Our team is responsible for the full object oriented analysis, requirements modeling, system design, and software implementation of the ATM system. We were tasked with creating a complete Use Case Model, specifying the system's required functionality, designing the entire software architecture, modeling its components using UML, and producing a working implementation in C++.

## **System Overview**

The ATM must support a range of financial transactions for MSB customers, including deposits, withdrawals, transfers, balance inquiries, and receipt generation. Customers access the system using an ATM card issued by MSB. These ATMs are connected to a centralized Oracle database located at MSB Headquarters, which stores all customer information, account data, loan limits, transaction histories, and authentication credentials. Every ATM transaction, successful or failed, is automatically logged in the central database.

The system must support five standard customer accounts Checking, Savings, Money Market, Consumer Loan, and Mortgage. Additionally, Certificate of Deposit (CD) accounts are available for balance inquiry only. MSB maintains a group of preferred customers who receive additional privileges, including no hold on deposits and overdraft protection through their Consumer Loan account. Non-preferred customers face specific restrictions. A 3-day hold on

check deposits and no overdraft protection, withdrawals may not exceed the account balance. Withdrawals must be in multiples of \$10 and limited to \$500 per day. Transfers may be made between Checking, Savings, and Money Market accounts. From Checking/Savings/Money Market to Consumer Loan and or from Checking/Savings/Money Market to Mortgage. The ATM must also support non-MSB customers, who may deposit or withdraw from their checking or savings accounts, with a service fee of \$3.50 per transaction.

## **Project Objectives**

### **1. Requirements and Specification**

We identified, and documented all ATM functionalities using use case diagrams, fully detailed use case descriptions, and operation highlighted system interactions.

### **2. System Design**

We produced UML class diagrams, communication diagrams, class descriptions, operation sequences, and scenario sequences that clearly define how the system's components interact. Our design ensures full traceability from requirements to implementation.

### **3. Implementation**

The final phase involves developing the software using C++. This includes implementing the system classes, methods, and logic for authentication, database interaction, transaction processing, error handling, and session control.

## **Purpose of the Final Document**

This combined document presents the final, fully integrated product of all three phases, requirements, design, and implementation, into one consistent structured system specification. It demonstrates how initial high level requirements evolved through analysis and design into a working software solution. The final result is a traceable, coherent, and complete ATM system model that MSB could implement to modernize its operations and provide improved banking services to its customers.

## **Team Information and Overview**

The project began with our professor assigning our class into teams of four, with the professor serving as project manager. Our first major milestone was the Requirements and Design phase, where we were responsible for producing UML use case diagrams, detailed dialogs, and the foundations of the system architecture. We were given two weeks to complete this phase.

As a team, we decided to divide the work so that each member would complete four use cases and contribute to the early design models. I completed my assigned use cases and additionally produced the initial class design and class descriptions. We planned to meet with our project manager the Monday before the deadline to review the work. However, only one of the three other group members had completed their portion. Because of the lack of completed material, there was not enough material to review.

This setback caused our team to be behind schedule. After discussing expectations with my assigned team, I continued working and completing an additional use case so that we would have enough material for submission. By Friday which was three days before the deadline, only two of us had completed roughly 75% of the project. Even though we repeatedly reminded the team of the deadline, the remaining 2 members submitted their work at 2:00 AM on Monday, leaving no opportunity for further review.

Their late work contained additional issues that created concerns, the formatting did not follow project specifications, diagrams appeared to be taken directly from generative AI, text styles and fonts were inconsistent, and the content did not follow guidelines given in class. These problems first occurred in Part 1 of the project. After meeting with the team and addressing the issue directly, the other members assured us that they would follow instructions properly for Part 2. However, when Part 2 was submitted, similar problems reappeared. Their work was produced using generative AI, improperly formatted, and inconsistent with the expected objectives. After raising these repeated concerns with our project manager, the decision was made to split the team into smaller groups to improve accountability and workflow.



After we split our teams into 2 separate teams, our workload increased for each member, however, the quality also improved significantly. My teammate and I were able to divide the work to be more efficient and have consistent progress through phases 3 and 4 of the project (Design and Implementation). In part 3 I initially struggled with creating communication diagrams and contributed fewer diagrams than planned, I was able to contribute more heavily in other areas. Specifically in later stages of the project and in the implementation and code development.

Overall, the decision to separate our teams improved our productivity, communication, and time management. Our team also agreed on redoing all parts of steps 1 and 2 to ensure consistency and accuracy before moving on to step 3. This decision increased our workload tremendously but this allowed us to have much more confidence in the overall system design. Even though we faced challenges at the beginning of the semester with the teams, the smaller team resulted in better contribution, better collaboration, and a stronger final product. This experience gave us a taste of real world software engineering and taught us an important lesson. Varying work ethics amongst team members may cause internal conflicts but it is always important to emphasize quality over quantity.

**Author: Dylan**

**Last Updated 10/29/2025**

### **Logon/Authentication**

Customers access ATMs using an ATM card issued by MSB.

### **Central Database**

All ATMs are connected to a central database at MSB Headquarters.

Every ATM transaction is automatically recorded in the central database.

The central database is maintained using Oracle DBMS and stores all customer information.

The central database handles interactions with other banks.

### **Customer Accounts**

ATM supports five customer account types: Checking, Savings, Money Market, Consumer Loan, and Mortgage.

Certificates of Deposit (CDs): Only balance inquiries; no transactions are allowed.

### **Deposits**

Customers can deposit money to Checking, Savings, or Money Market accounts.

Preferred customers: No hold on deposits.

Other customers: 3-day hold on check deposits only.

Only cash (paper money) and checks are accepted.

### **Withdrawals**

Customers can withdraw money from Checking, Savings, or Money Market accounts.

Non-preferred customers cannot withdraw more than their account balance (no overdraft).

Customers may withdraw from Consumer Loan accounts up to their loan limit.

Preferred customers may overdraft from their accounts; excess funds are drawn from their Consumer Loan account.

Daily withdrawal limit: \$500, in multiples of \$10.

**Transfers**

Customers can transfer funds:

Between Checking, Savings, and Money Market accounts.

From Checking, Savings, or Money Market to Consumer Loan.

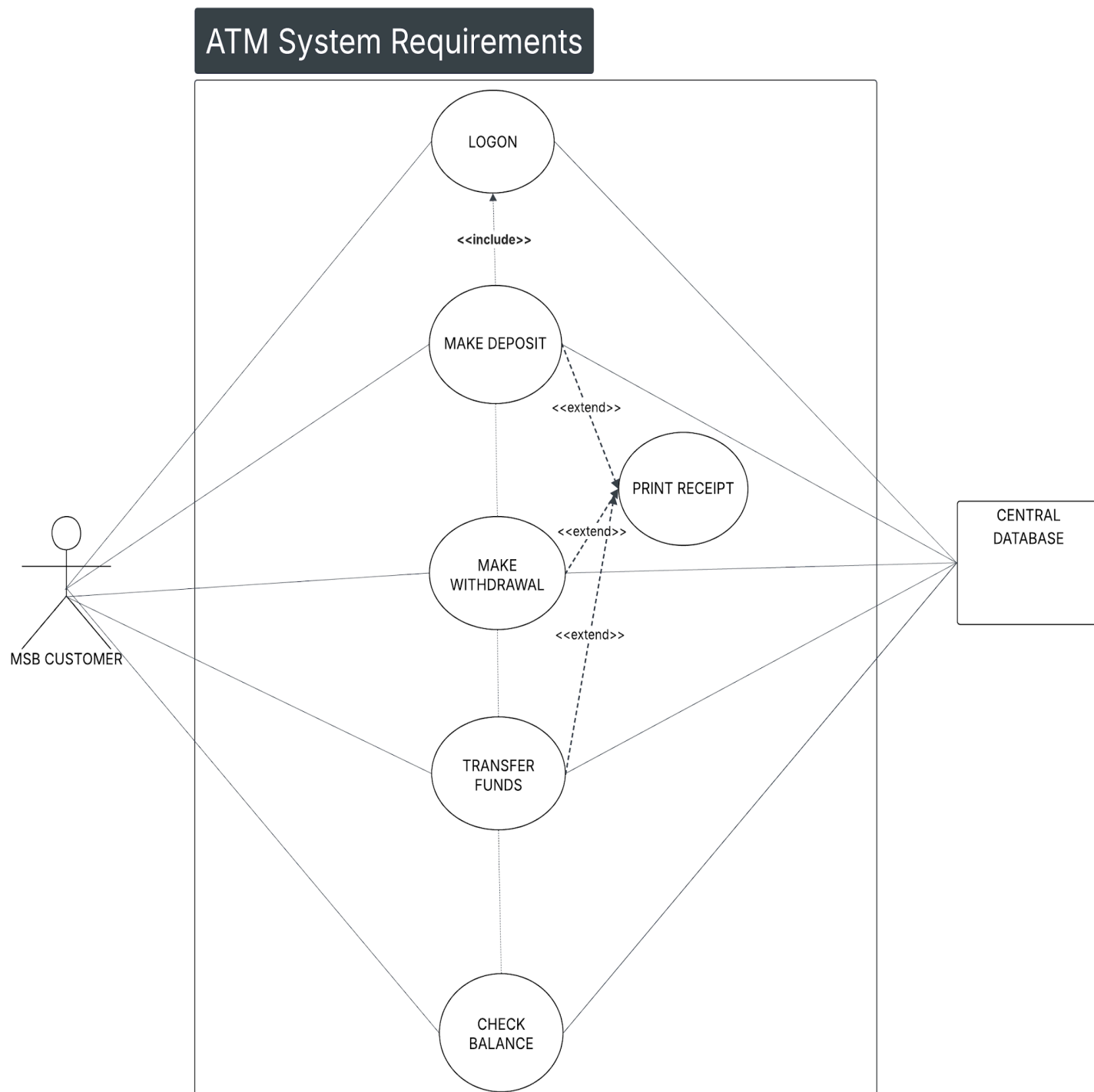
From Checking, Savings, or Money Market to Mortgage.

**Balance Inquiry**

Customers can check balances for all accounts (Checking, Savings, Money Market, CD, Consumer Loan, Mortgage).

**Receipts**

Transaction receipts may be printed; customers can choose to decline.

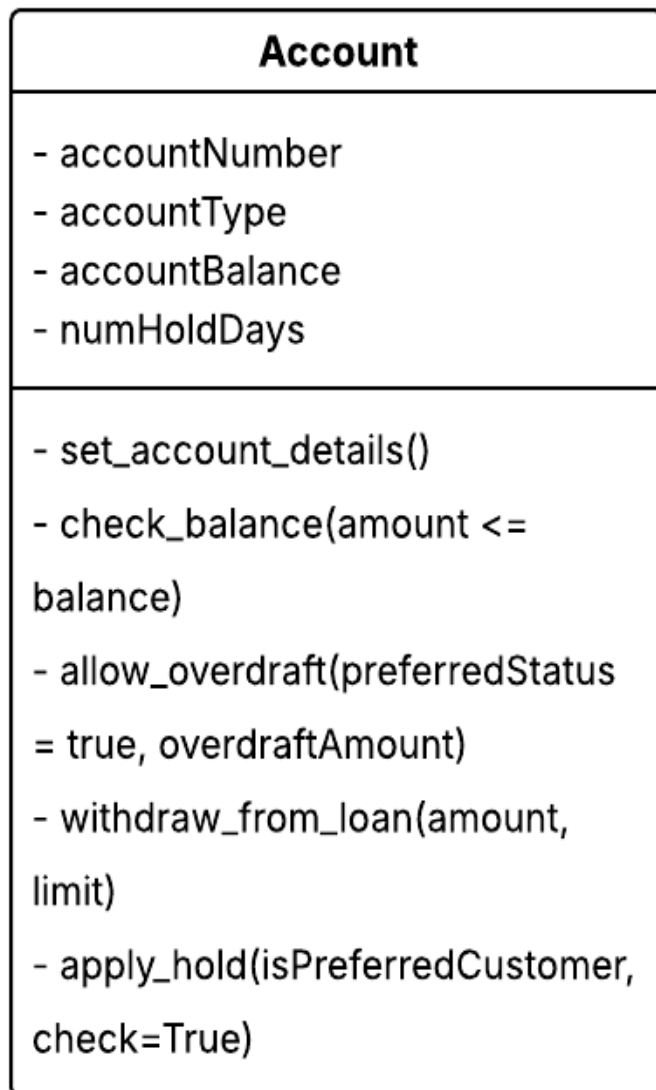
**ATM USE CASE DIAGRAM****AUTHOR: DE-GRAFT QUANSAH****LAST UPDATE: 12/06/2025**

## ATM CLASS DIAGRAMS AND DESCRIPTIONS

**AUTHOR: DE-GRAFT QUANSAH**

**LAST UPDATE: 12/06/2025**

**[Account]:** The Account class represents any customer-owned account connected to the ATM system (Checking, Savings, Money Market, Consumer Loan, Mortgage, or CD). It stores balance information, account type, overdraft status, and other basic attributes. This class provides operations like `check_balance()`, `allow_overdraft()`, and `select_type()`, enabling the ATM to verify funds, apply overdraft rules for preferred customers, and identify which account is being accessed during a transaction





**[CDAccount]:** The CDAccount subclass models a Certificate of Deposit account. Since MSB's ATM rules do not allow deposits, withdrawals, or transfers for CDs, this class is limited. Its single function, `check_balance()`, lets the customer view the current value of their CD without performing any financial transactions. The class exists mainly to provide read-only balance inquiry capability, with all other actions restricted to maintain the rules defined by the bank.

| CDAccount         |
|-------------------|
| (inherited)       |
| - check_balance() |

**[CentralDatabase]:** The CentralDatabase class acts as the authoritative storage system for all customer information and account data. It maintains balances, user credentials, loan limits, daily transaction totals, and transaction histories. The class exposes operations such as `authenticate_user()`, `update_balance()`, `get_account_details()`, and `record_transaction()`, which the ATM relies on for accurate, real-time financial processing.

| CentralDatabase                                                                                                                                                                                                                                                                                    |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>- customerInfo</li><li>- transactionLog</li></ul>                                                                                                                                                                                                            |
| <ul style="list-style-type: none"><li>- authenticate_user(ID, PW, OUT status)</li><li>- get_user_info()</li><li>- get_account_details()</li><li>- update_balance()</li><li>- record_transaction()</li><li>- store_customer_info()</li><li>- interact_with_banks()</li><li>- send_email()</li></ul> |

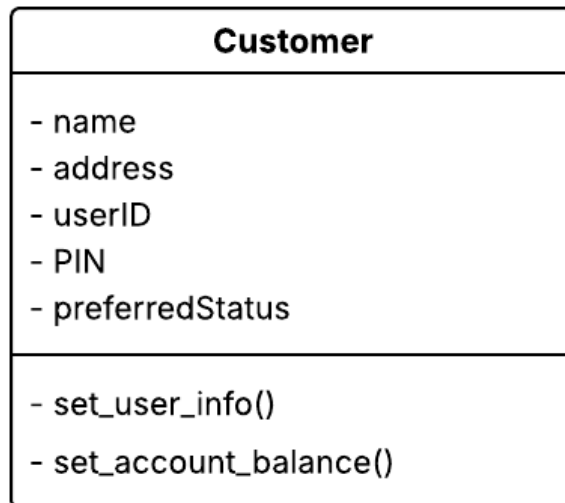
**[CheckingAccount]:** The CheckingAccount subclass represents the user's checking account and inherits the core attributes and behaviors from the main Account class. Its primary purpose is to support overdraft protection for preferred customers. The class provides the `allow_overdraft()` function, which determines whether the account can temporarily access additional funds from the customer's Consumer Loan account when the checking balance is not sufficient. Other checking-related behavior is managed through the parent Account class, making `allow_overdraft()` the central responsibility of this subclass.

| CheckingAccount     |
|---------------------|
| (inherited)         |
| - allow_overdraft() |

**[ConsumerLoanAccount]:** The ConsumerLoanAccount subclass represents a customer's personal loan account. Its primary purpose is to handle loan withdrawals either directly (when the customer chooses to withdraw from a loan) or indirectly through overdraft protection. The class provides operations like `withdraw_from_loan()` to ensure that loan limits are respected and any overdraft usage is properly tracked.

| ConsumerLoanAccount                    |
|----------------------------------------|
| (inherited)                            |
| - withdraw_from_loan(amount,<br>limit) |

**[Customer]:** The Customer class stores information related to the ATM user, including ID, name, preferred-customer status, linked accounts, daily withdrawal totals, and authentication results. It interacts closely with the CentralDatabase to load and update customer-specific information during a session, using operations like `set_user_info()` and `set_account_balance()`.



**[ErrorHandling]:** The ErrorHandling class manages all incorrect or exceptional situations that occur during ATM operations. It handles invalid deposits, insufficient funds, overdraft failures, authentication problems, and session termination logic. This class centralizes user-facing recovery steps using functions like `handle_invalid_credentials()`, `cancel_transaction()`, `handle_insufficient_funds()`, and `terminate_session()`.

| ErrorHandling                                                                                                                                                                                                                                                                                                                                                                                                                            |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| <ul style="list-style-type: none"><li>- <code>handle_invalid_credentials()</code></li><li>- <code>handle_invalid_deposit()</code></li><li>- <code>cancel_transaction()</code></li><li>- <code>terminate_session()</code></li><li>- <code>terminate_transaction()</code></li><li>- <code>handle_overdraft_limit_reached()</code></li><li>- <code>handle_insufficient_funds()</code></li><li>- <code>handle_print_error()</code></li></ul> |

**[MSB\_ATM]:** The MSB\_ATM class represents the physical ATM machine and its core operational logic. It coordinates user interactions, triggers transactions, manages cash dispensing or deposit capturing, and directs the flow between customer actions and backend operations. Example of the responsibilities of this class include `dispense_cash()`, `print_receipt()`, `capture_deposit()`, `logout()`, and communicate with the CentralDatabase.

| MSB_ATM                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>- location</li><li>- managedBy</li></ul>                                                                                                                                                                                                                                                                                                                                                                                               |
| <ul style="list-style-type: none"><li>- communicate_DB()</li><li>- logout()</li><li>- complete_transaction()</li><li>- capture_deposit()</li><li>- set_withdrawal_amount()</li><li>- check_available_funds()</li><li>- return_funds()</li><li>- cancel_deposit()</li><li>- validate_deposit()</li><li>- confirm_deposit()</li><li>- send_user_response()</li><li>- dispense_cash()</li><li>- select_transfer()</li><li>- check_printer()</li><li>- print_receipt()</li></ul> |

**[Screen]:** The Screen class controls all user interface elements of the ATM. Its purpose is to display menus, account lists, prompts, confirmation messages, errors, and transaction summaries. Operations like `display_logon_screen()`, `display_account_types()`, `display_error()`, `display_print_receipt()`, and `display_transfer_summary()` allow customers to interact with the ATM in a guided and understandable way.

| Screen                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"> <li>+ <code>display_logon_screen()</code></li> <li>+ <code>capture_ID_PW()</code></li> <li>+ <code>display_system_functions()</code></li> <li>+ <code>display_service_menu()</code></li> <li>+ <code>display_withdrawal_accounts()</code></li> <li>+ <code>display_service_fee()</code></li> <li>+ <code>display_invalid_credentials()</code></li> <li>+ <code>display_account_types()</code></li> <li>+ <code>display_account_details()</code></li> <li>+ <code>display_limit_exceeded()</code></li> <li>+ <code>display_insufficient_funds()</code></li> <li>+ <code>display_source_accounts()</code></li> <li>+ <code>display_error()</code></li> <li>+ <code>display_deposit_cancelled()</code></li> <li>+ <code>display_invalid_deposit()</code></li> <li>+ <code>display_invalid_transfer()</code></li> <li>+ <code>display_print_receipt()</code></li> <li>+ <code>capture_amount()</code></li> <li>- <code>send_user_selection()</code></li> <li>+ <code>display_transfer_failed()</code></li> <li>+ <code>display_transfer_summary()</code></li> <li>+ <code>capture_email()</code></li> </ul> |

**[Transaction]:** The Transaction class handles the logic behind all ATM financial actions, such as withdrawals, deposits, transfers, and balance inquiries. It ensures each transaction follows MSB rules (e.g., daily limits, overdraft behavior, deposit holds). It includes methods like `withdraw_funds()`, `deposit_funds()`, `transfer_funds()`, `apply_hold()`, `process_transaction()`, and `log_transaction()`.

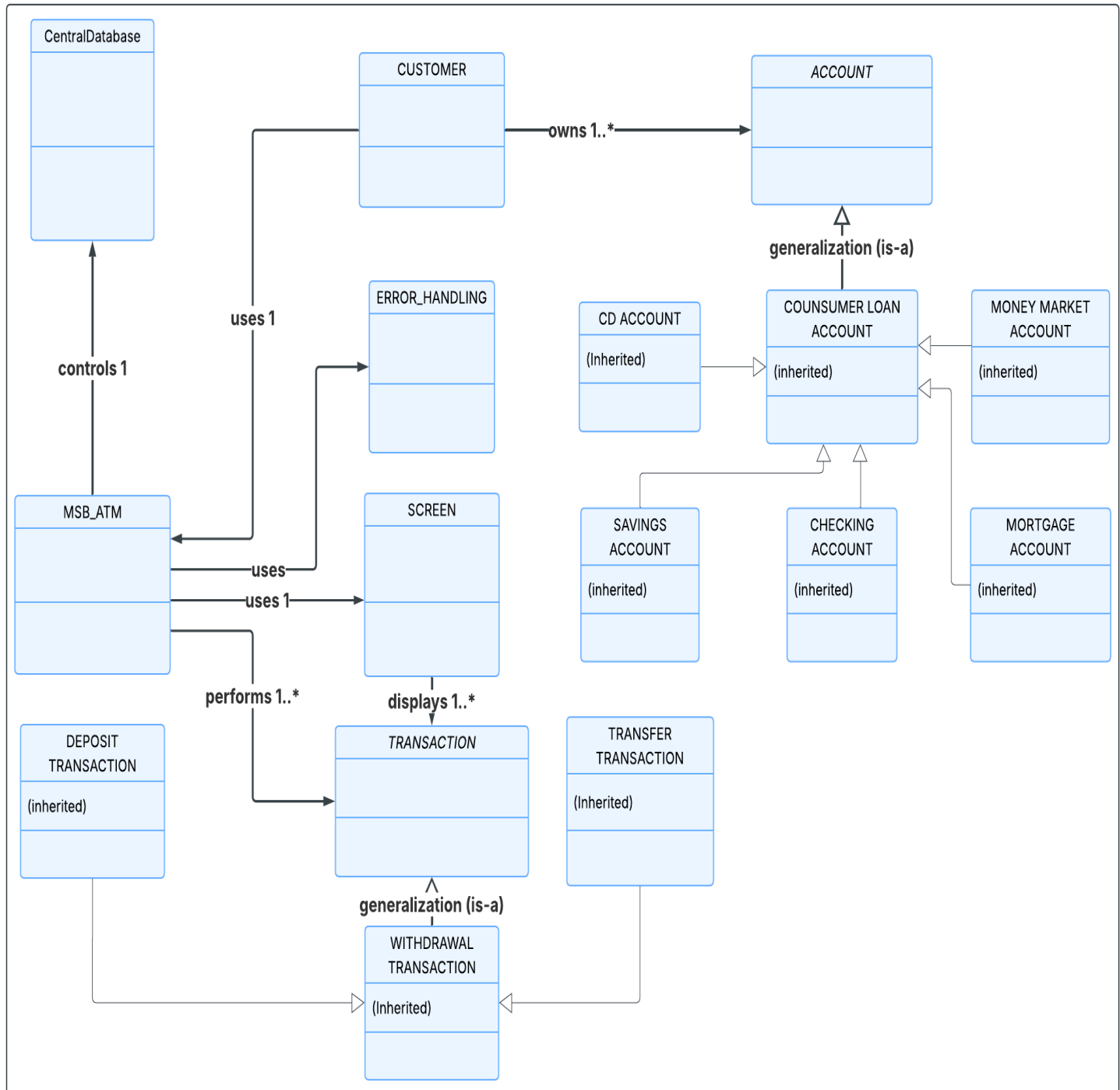
| Transaction                                                                                                                                                                                                                                                                               |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>#transactionID</code><br><code>#amount</code><br><code>#timeStamp</code><br><code>#transactionType</code><br><code>#serviceFee</code>                                                                                                                                               |
| <ul style="list-style-type: none"><li>- <code>withdraw_funds()</code></li><li>- <code>check_limit()</code></li><li>- <code>deposit_funds()</code></li><li>- <code>transfer_funds()</code></li><li>- <code>validate_amount()</code></li><li>- <code>process_transaction()</code></li></ul> |



## ATM STATIC RELATIONSHIP DIAGRAM

AUTHOR: DE-GRAFT QUANSAH LAST UPDATE: 12/06/202

### ATM Class Relationship Diagram



**Author: Dylan Weaver**

**Last Update: 12/06/2025**

**Use Case: LOGON**

**Pre-Conditions:**

- The user (Customer, System Administrator, or Manager) of MSB ATM is signed out.
- The user inserts their card. The ATM displays the welcome logon screen.

**Dialog:**

1. The user inserts their card, and the welcoming logon screen **appears**.
  - [Screen]:display\_logon\_screen()
2. The logon screen **prompts** the user to enter their **Password (PIN)**.
  - [Screen]:capture\_ID\_PW().
  - For non-MSB customers, the system **displays** a message notifying the user that a **\$3.50 service fee** applies per transaction. [Screen]: display\_service\_fee()
  - **User Declines Fee:** The session is **terminated** and the card is ejected.  
[ErrorHandling]:terminate\_session()
  - If the user accepts the fee, the welcome logon screen **appears** and prompts the user to **select** their preferred account type. [Screen]:display\_account\_types()
3. The system **communicates** with the **Central Database** via the interface to **authenticate** the user's credentials. [MSB\_ATM]:communicate\_DB(),  
[CentralDatabase]:authenticate\_user(ID, PW) -> OUT status
  - Based on the user ID and Password, the ATM System **communicates** with the Central Database (via DATABASE INTERFACE) and identifies user type and related functions (if [CentralDatabase]: OUT status = "validated")
  - **Invalid Credentials:** The system **displays** an error message and prompts the user to re-enter valid credentials. (if [CentralDatabase]: OUT status != "validated"),  
[ErrorHandling]:handle\_invalid\_credentials(),  
[Screen]:display\_invalid\_credentials()

- Allow users to enter ID and/or Password a maximum of 3 times. After 3 invalid attempts, the **session terminates** and the card is ejected, and screen returns to logon. `[ErrorHandling]:invalid_credentials()`,  
`[ErrorHandling]:terminate_session()`, `[Screen]:display_logon_screen()`
- 4. Upon successful authentication, the system **identifies** the customer type and grants access to corresponding functionalities:
  - During authentication, the ATM System **retrieves** the user data from the Central Database and **sets** user data while the session is active:  
`[MSB_ATM]:communicate_DB()`, `[CentralDatabase]:get_user_info()`,  
`[Customer]:set_user_info()`, `[CentralDatabase]:get_account_balance()`,  
`[Customer]:set_account_balance()`

**Customer:** Accesses ATM services such as deposit, withdrawal, transfer, and balance inquiry.

**System Administrator:** Accesses system maintenance and monitoring functions.

**Manager:** Accesses administrative or reporting functionalities.

#### Postconditions:

- The user is successfully authenticated and **redirected** to their authorized ATM or system functions. `[Screen]:display_system_functions()`
- User session **terminates** normally `[MSB_ATM]:logout()`
- User session **terminates** after invalid attempts  
`[ErrorHandling]:handle_invalid_credentials()`, `ErrorHandling:terminate_session()`
- Upon successful login, based on registered user ID, the system **displays** the corresponding ATM service menu `[Screen]:display_service_menu()`

#### Multiplicity:

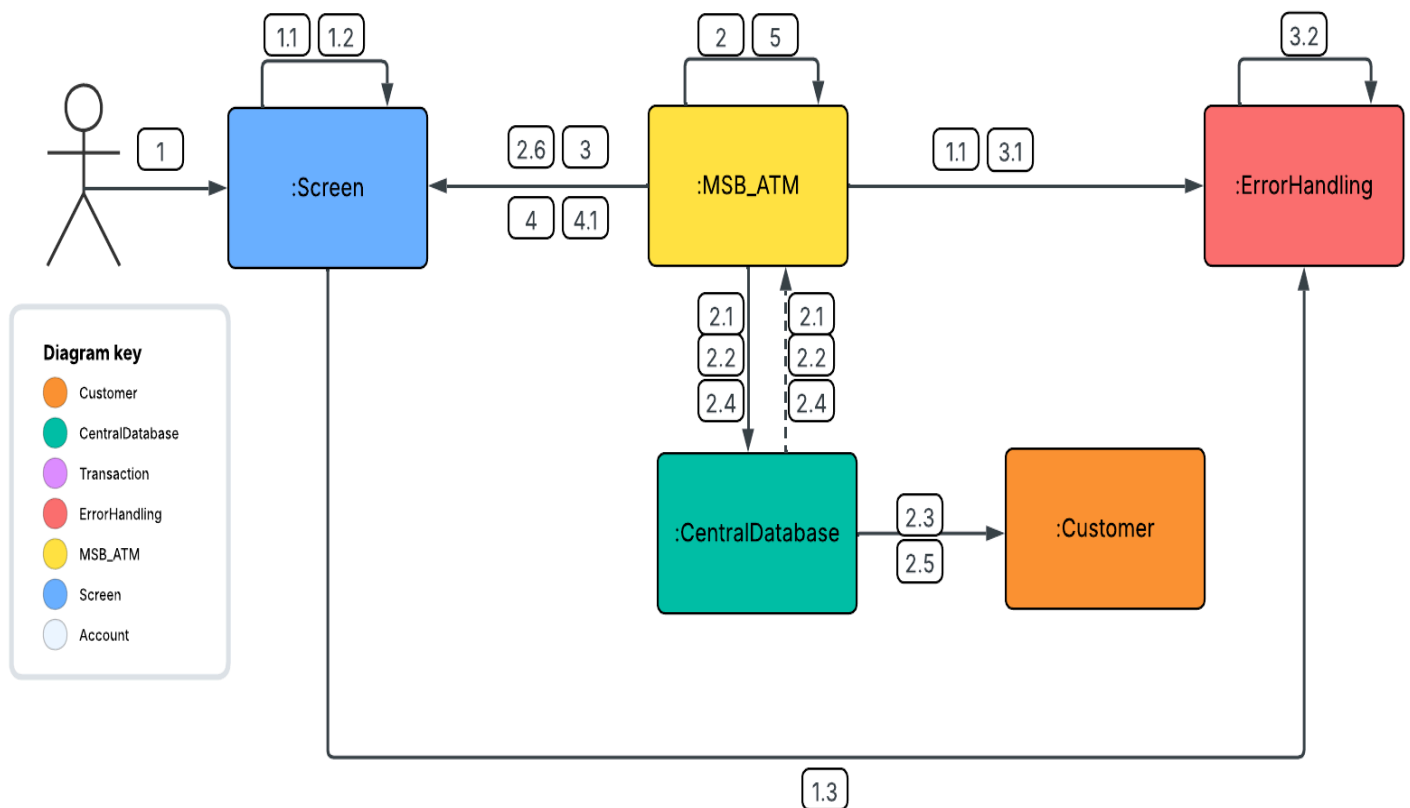
Each Logon session involves exactly:

- 1 `MSB_ATM` object (the machine performing the login)
- 1 `Customer` object (the user attempting to log in)
- 1 `CentralDatabase` object (verifying credentials & loading data)
- 1 `Screen` object (displaying login prompts)
- 1 `ErrorHandling` object (if invalid credentials occur)

**Author: De-Graft Quansah**

**Last Update: 12/06/2025**

### Communication Diagram



### Operation sequence

- 1: **display\_logon\_screen()**
- 1.1: **capture\_ID\_PW()**
- 1.2: **display\_service\_fee** (only for non-MSB customers)
- 1.3: **terminate\_session()** {User Declines Fee}
- 2: **communicate\_DB()**

- 2.1: `authenticate_user(ID, PW)` -> OUT status
- 2.2: `get_user_info()` (If status = validated)
- 2.3: `set_user_info()`
- 2.4: `get_account_balance()`
- 2.5: `set_account_balance()`
- 2.6: `display_account_types()` (If validated and fee accepted)
- 3: `display_invalid_credentials()` (If OUT status != validated)
  - 3.1: `handle_invalid_credentials()` (count < 3 → retry)
  - 3.2: `terminate_session()` (count = 3 → card ejected, return to login)
- 4: `display_system_functions()` (Upon validated login)
  - 4.1: `display_service_menu()` (ATM-specific services based on user role)
- 5: `logout()`

## Scenario Sequence

1, 1.1 , 2 , 2.1 , 2.2 , 2.3 , 2.4 , 2.5 , 2.6 , 4 , 4.1 – **Successful Login (MSB-Customer)**

1 , 1.1 , 1.2 , 2 , 2.1 , 2.2 , 2.3 , 2.4 , 2.5 , 2.6 , 4 , 4.1 – **Successful Login (Non-MSB-Customer Accepts Fee)**

1, 1.1, 1.2, 1.3 – **Non-MSB Customer Declines Fee**

1, 1.1, 3, 3.1, 3.2 – **Invalid Credentials (Max Attempts Reached)**

5 – **System Timeout / Auto Logout**

**Author: De-Graft**

**Last Update: 12/06/2025**

## **Use Case: MAKE WITHDRAWAL**

### **Pre-Conditions:**

- The user is already **authenticated**. `[CentralDatabase]:authenticate_user()`
- User session is active and account information is **loaded**.  
`[CentralDatabase]:get_user_info(), [CentralDatabase]:get_user_info(),`  
`[Customer]:set_user_info(), [Customer]:set_account_balance()`
- Withdrawal must follow MSB rules:
  - Daily withdrawal limit is \$500, across all accounts.
  - Withdrawal amount must be in multiples of \$10.
  - Only Checking, Savings, Money Market, and Consumer Loan accounts support withdrawals.
  - Mortgage and CD accounts cannot be withdrawn from.
- Non-MSB customers must accept the standard \$3.50 service fee before any withdrawal is allowed.

### **Dialog:**

1. The ATM **displays** all accounts that allow withdrawals (Checking, Savings, Money Market, Consumer Loan). `[Screen]:display_withdrawal_accounts()`
  - If the user is a non-MSB customer, the ATM reminds them that a **\$3.50 fee applies** to the transaction. `[Screen]:display_service_fee()`
  - If the user declines, the withdrawal is cancelled.  
`[ErrorHandling]:terminate_session()`
2. The ATM **prompts** the user to enter a withdrawal amount. `[Screen]:capture_amount()`
  - The ATM **checks** that the entered amount is a multiple of \$10.  
`[Transaction]:validate_amount(amount, $10)`
  - If invalid, **prompt** the user to re-enter. `[ErrorHandling]:handle_invalid_amount(),`  
`Screen]:display_invalid_amount()`

3. The ATM system **ensures** the user's total withdrawals today (including this ongoing request) does not exceed the daily limit of \$500. `[Transaction]:check_limit(dailyTotal, $500)`
  - If the daily limit is exceeded, the withdrawal is stopped.  
`[Screen]:display_limit_exceeded(), [ErrorHandling]:terminate_transaction()`
4. The system **checks** the user and the account type and applies the respective Account functions.
  - Non-preferred customers **cannot overdraft**. `[Account]:check_balance(amount <= balance)`.
    - If insufficient funds: `[Screen]:display_insufficient_funds(), [ErrorHandling]:terminate_transaction()`
  - Preferred customers may **overdraft** from their Checking, Savings, or Money Market accounts. `[Account]:allow_overdraft(preferredStatus = true, overdraftAmount)`. Any overdraft is automatically pulled from the Consumer Loan Account (up to loan limit).
    - User may withdraw only up to their remaining loan limit.  
`[ConsumerLoanAccount]:withdraw_from_loan(amount, limit)`
5. The ATM **checks** if it physically has enough cash to dispense.  
`[MSB_ATM]:check_available_funds(amount)`
  - If ATM has insufficient cash: `[Screen]:display_insufficient_funds(), [ErrorHandling]:terminate_transaction()`
6. The ATM **processes** the withdrawal request, **updates balances**, and **saves** a record with the central database. `[MSB_ATM]:communicate_DB(), [CentralDatabase]:update_balance(), [CentralDatabase]:record_transaction()`
7. The ATM **dispenses** the requested cash. `[MSB_ATM]:dispense_cash()`
8. The user is given the option to **print a receipt** confirming the deposit. `[Screen]:display_print_receipt(), [MSB_ATM]:print_receipt()`

**Post-Conditions:**

- The user's account balance is **updated** in the Central Database.  
[CentralDatabase]:update\_balance()
- The daily withdrawal total and the transaction is **recorded**.  
[CentralDatabase]:record\_transaction()
- Cash is **dispensed** and the withdrawal is considered complete.  
[MSB\_ATM]:dispense\_cash()
- Control **returns** to the ATM menu unless the user ends the session.  
[MSB\_ATM]:display\_service\_menu()

**Multiplicity:**

A withdrawal transaction is linked with

- 1 **MSB\_ATM** object (dispensing cash & guiding user)
- 1 **Customer** object (the authenticated user)
- 1 **Account** object (the withdrawal account)
- 0 or 1 **ConsumerLoanAccount** object (only used for overdraft for preferred customers)
- 1 **CentralDatabase** object (updating balances and transaction history)
- 1 **Screen** object (displaying prompts and results)
- 1 **Transaction** object (handling withdrawal logic)
- 1 **ErrorHandling** object (if insufficient funds, incorrect amount, etc.)





- 2: `capture_amount()`
- 2.1: `validate_amount()` (multiple of \$10)
- 2.2: `handle_invalid_amount()`
- 2.3: `display_invalid_amount()`
- 3: `check_limit(dailyTotal, $500)`
- 3.1: `display_limit_exceeded()`
- 4: `check_balance()`
- 4.1: `allow_overdraft()` (Preferred users)
- 4.2: `withdraw_from_loan()` (Consumer Loan)
- 4.3: `display_insufficient_funds()`
- 4.4: `terminate_transaction()`
- 5: `check_available_funds()`
- 5.1: `display_insufficient_funds()`
- 6: `communicate_DB()`
- 6.1: `update_balance()`
- 6.2: `record_transaction()`
- 7: `dispense_cash()`
- 8: `display_print_receipt()`
- 8.1: `print_receipt()`
- 9: `display_service_menu()`

### 3. Scenario Sequences

1 , 2 , 2.1 , 3 , 4 , 5 , 6 , 7 , 8 , 9 – **Successful Withdrawal (MSB-Customer, Sufficient Funds, No Receipt)**

1 , 2 , 2.1 , 3 , 4 , 5 , 6 , 7 , 8 , 8.1, 9 – **Successful Withdrawal (MSB-Customer, Sufficient Funds, Receipt Printed)**

1 , 1.1 , 2 , 2.1 , 3 , 4 , 5 , 6 , 7 , 8 , 9 - **Successful Withdrawal (Non-MSB Customer — Fee Accepted)**

1, 1.1, 1.2 – **Non-MSB Customer Declines Fee**

1, 2 , 2.1 , 2.2 , 2.3, 2 (retry) -**Invalid Amount (Not Multiple of \$10)**

1, 2 , 3 , 3.1 , 4.4 - **Customer Exceeds Daily Limit**

1 , 2 , 2.1 , 3 , 4 (insufficient balance detected) , 4.1 , 4.2 , 5 , 6 , 7 , 8 , 9 - **Insufficient Funds (Preferred customer. Overdraft Protection)**

1, 2, 2.1, 3, 4 , 4.3 , 4.4 - **Insufficient Funds (Non-Preferred Customer. No Overdraft Protection)**

5 , 5.1 , 4.4 - **ATM Out of Cash**

**Author: Dylan Weaver**

**Last Update: 12/06/2025**

## **Use Case: MAKE DEPOSIT**

### **Pre-Conditions:**

- User is authenticated. `[CentralDatabase]:authenticate_user()`
- User session is active and the ATM has already retrieved the user's account information.  
`[CentralDatabase]:get_user_info()`, `[Customer]:set_user_info()`,  
`[CentralDatabase]:get_account_balance()`, `[Customer]:set_account_balance`
- Deposits are only allowed into Checking, Savings, or Money Market accounts.
- CD and Mortgage accounts do not support deposits.
- Non-MSB customers must accept the standard \$3.50 service fee before making a deposit.

### **Dialog:**

1. Customers can **make deposits** into their *Checking, Savings, or Money Market accounts*.  
The ATM **displays** the deposit-eligible account list. `[Screen]:display_account_options()`
  - If the user is a non-MSB customer, the ATM **informs** them that a \$3.50 fee applies to the transaction. `[Screen]:display_service_fee()`
    - If the user declines the fee, the deposit is cancelled.  
`[ErrorHandling]:terminate_session()`
2. After choosing an account, the ATM **displays** the selected account details.  
`[Screen]:display_account_details()`
3. The system instructs the user to **insert** cash or checks into the deposit slot.  
`[MSB_ATM]:capture_deposit()`
  - The ATM **validates** the deposit (e.g., unreadable checks, torn bills, empty envelope, etc.).`[Transaction]:validate_deposit()`
  - **Invalid Deposit:**
    - The ATM **returns** whatever was inserted and displays an error message.  
`[ErrorHandling]:handle_invalid_deposit()`, `[MSB_ATM]:return_funds()`,  
`[Screen]:display_invalid_deposit()`
  - If the user **cancels** during the deposit process: `[MSB_ATM]:cancel_deposit()`,  
`[ErrorHandling]:terminate_transaction()`, `[MSB_ATM]:return_funds()`,  
`[Screen]:display_deposit_cancelled()`
4. The ATM **shows** the counted amount and asks the user to confirm.  
`[Screen]:display_confirm_deposit()`, `[MSB_ATM]:confirm_deposit()`
5. The ATM **determines** whether any portion of the deposit requires a hold.
  - Rules from MSB:

- Cash → No hold applied for everyone
- Preferred customers → No hold applied on checks
- Non-preferred customers depositing checks → 3-day hold applied

`[Account]:apply_hold(isPreferred, isCheckDeposit)`

6. The ATM **processes** the deposit and **updates balances and records** transaction.  
`[MSB_ATM]:communicate_DB()`, `[CentralDatabase]:update_balance()`,  
`[CentralDatabase]:record_transaction()`
7. The user is given the option to **print or display a receipt** confirming the deposit.
  - `[Screen]:display_print_receipt("Print receipt")`
  - `[MSB_ATM]:print_receipt()`

#### Post-Conditions:

- The deposit amount (minus any holds) is credited to the user's account.  
`[CentralDatabase]:update_balance()`
- Deposit activity is **recorded** as a transaction. `[CentralDatabase]:record_transaction()`
- A receipt is **printed** based on the user's selection. `[MSB_ATM]:print_receipt()`

#### Multiplicity:

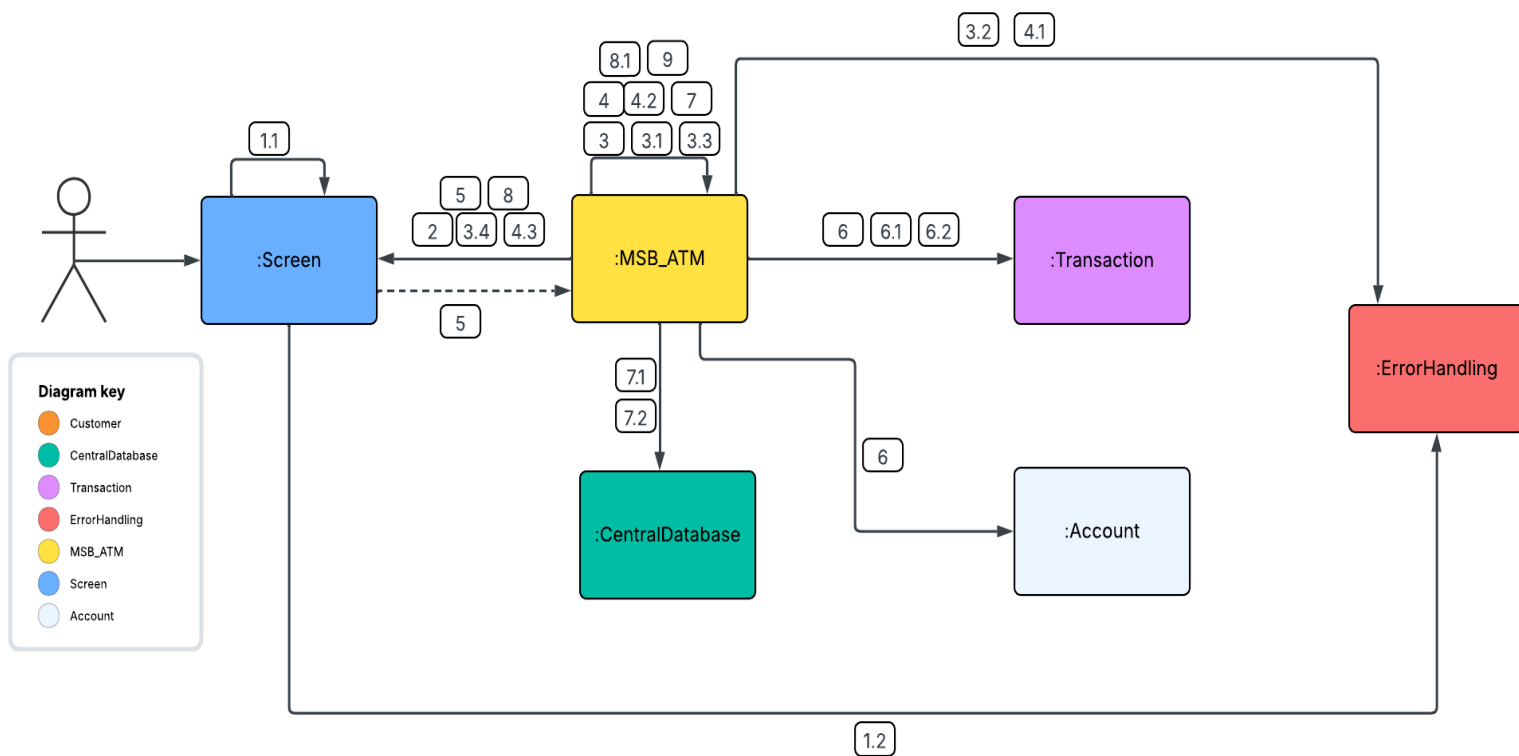
A deposit transaction is linked with

- 1 **MSB\_ATM** object (capturing deposit & printing receipt)
- 1 **Customer** object
- 1 **Account** object (Checking, Savings, or Money Market. ConsumerLoan or Mortgage accounts do not participate in deposits.)
- 1 **Transaction** object
- 1 **CentralDatabase** object
- 1 **Screen** object
- 1 **ErrorHandling** object (invalid deposit, cancelled deposit)

**Author: De-Graft Quansah**

**Last Update: 11/24/2025**

## Communication Diagram



## Operation Sequence

- 1: **display\_account\_options()**
- 1.1: **display\_service\_fee()** (Non-MSB only)
- 1.2: **terminate\_session()** (User declines fee)
- 2: **display\_account\_details()**
- 3: **capture\_deposit()**
- 3.1: **validate\_deposit()**

3.2: `handle_invalid_deposit()`  
3.3: `return_funds()`  
3.4: `display_invalid_deposit()`  
4: `cancel_deposit()` (If user cancels)  
4.1: `terminate_transaction()`  
4.2: `return_funds()`  
4.3: `display_deposit_cancelled()`  
5: `display_confirm_deposit()`  
6: `apply_hold()`  
7: `communicate_DB()`  
7.1: `update_balance()`  
7.2: `record_transaction()`  
8: `display_print_receipt()`  
8.1: `print_receipt()`  
9: `logout()`

## Scenario Sequence

1, 2, 3, 3.1, 5, 7, 8, 9 – **Successful Cash Deposit (MSB-Customer)**

1, 1.1, 2, 3, 3.1, 5, 7, 8, 9 – **Successful Cash Deposit (Non-MSB-Customer)**

1, 2, 3, 3.1, 5, 7, 8, 9 – **Successful Check Deposit (Preferred Customer)**

1, 2, 3, 3.1, 5, 6, 7, 8, 9 – **Successful Check Deposit (MSB & Non-Preferred Customer)**

1, 1.1, 2, 3, 3.1, 5, 6, 7, 8, 9 – **Successful Check Deposit (Non-MSB & Non-Preferred Customer)**

1, 1.1, 1.2 – **Non-MSB Customer Declines Fee**

1, 2, 3, 3.1, 3.2, 3.3, 3.4 – **Invalid Deposit (Unreadable Check, Empty Envelope, etc.)**

1, 2, 3, 4, 4.1, 4.2, 4.3 – **User Cancels Deposit Before Completion**

9 – **System Timeout / Auto Logout**



**Author Name: Dylan Weaver**

**Update: 12/06/2025**

**Use Case: TRANSFER FUNDS**

**Pre-Conditions:**

- User is **authenticated**.`[CentralDatabase]:authenticate_user()`
- User session is active and account information is already **loaded**.  
`[CentralDatabase]:get_user_info(), [CentralDatabase]:get_account_balance()`
- Transfer rules based on MSB requirements:
  - Transfers among Checking, Savings, Money Market accounts are allowed.
  - Transfers from Checking/Savings/Money Market to Consumer Loan (loan payment) are allowed.
  - Transfers from Checking/Savings/Money Market to Mortgage are allowed.
  - Transfers into CDs are NOT allowed (CD is balance-inquiry only).
  - Transfers from Consumer Loan or Mortgage to any account are NOT allowed.
- Non-MSB customers must accept the \$3.50 service fee before performing a transfer.

**Dialog:**

1. The user **selects** “Transfer Funds” from the main menu.`[MSB_ATM]:select_transfer()`
  - The ATM **displays** eligible (source) accounts to transfer from: Checking, Savings, or Money Market. `[Screen]:display_source_accounts(), [CentralDatabase]:get_account_details()`
  - If the user is a non-MSB customer, the ATM **shows** a \$3.50 transaction fee message.`[Screen]:display_service_fee()`
    - If declined, the system returns the user to the logon screen:  
`[ErrorHandling]:terminate_session(), [Screen]:display_logon_screen()`
2. The ATM **shows** a list of all transferable destination accounts allowed based on MSB rules.`[Screen]:display_account_types()`
3. The ATM **prompts** the user to enter the desired transfer amount.`[Screen]:capture_amount()`

4. The ATM **validates** that the source account has enough money for the transfer.

`[Account]:check_balance(amount <= balance)`

- If insufficient funds (non-preferred customer), user may re-enter amount or cancel. `[ErrorHandling]:handle_insufficient_funds()`,  
`[Screen]:display_insufficient_funds()`
- If the user is a preferred customer and balance is insufficient, overdraft protection may apply. `[Account]:allow_overdraft(preferredStatus = true, overdraftAmount)`
- If overdraft is needed, funds are pulled from the Consumer Loan account automatically.

`[ConsumerLoanAccount]:withdraw_from_loan(amount, limit)`

- If overdraft limit is reached:

`[ErrorHandling]:handle_overdraft_limit_reached()`,

`[Screen]:display_transfer_failed()`

- If still insufficient after overdraft:

`[ErrorHandling]:handle_insufficient_funds()`,

`[Screen]:display_transfer_failed()`

5. If no issues remain, the ATM prepares to **process** the transfer.

`[Transaction]:process_transaction()`.

- The ATM credits the destination account. `[Transaction]:transfer_funds()`,

6. The ATM **shows** a summary that includes:

- Source (withdrawal) account
- Destination (deposit) account
- Transfer amount

`[Screen]:display_transfer_summary()`

7. If the user cancels here, no changes are applied. `[ErrorHandling]:terminate_transaction()`

8. The ATM updates both accounts. `[MSB_ATM]:communicate_DB()`,

`[CentralDatabase]:update_balance()`, `[CentralDatabase]:record_transaction()`

9. When the transaction is **complete**, the ATM **asks** if the user would like a receipt or display it on screen. `[Screen]:display_print_receipt()`, `[MSB_ATM]:print_receipt()`

10. The ATM returns to the service menu or logs the user out if the session is ending.

[MSB\_ATM]:logout(), [ErrorHandling]:terminate\_session(),  
[Screen]:display\_logon\_screen()

#### Post-Conditions:

- Funds are successfully **transferred** between the selected accounts. [Transaction]:transfer\_funds(), [CentralDatabase]:update\_balance()
- The transaction is **recorded**. [CentralDatabase]:record\_transaction()
- A receipt is **printed** or shown on screen, based on user preference.  
[MSB\_ATM]:print\_receipt()
- User is **returned** to the main menu or logged out. [MSB\_ATM]:logout()

#### Multiplicity:

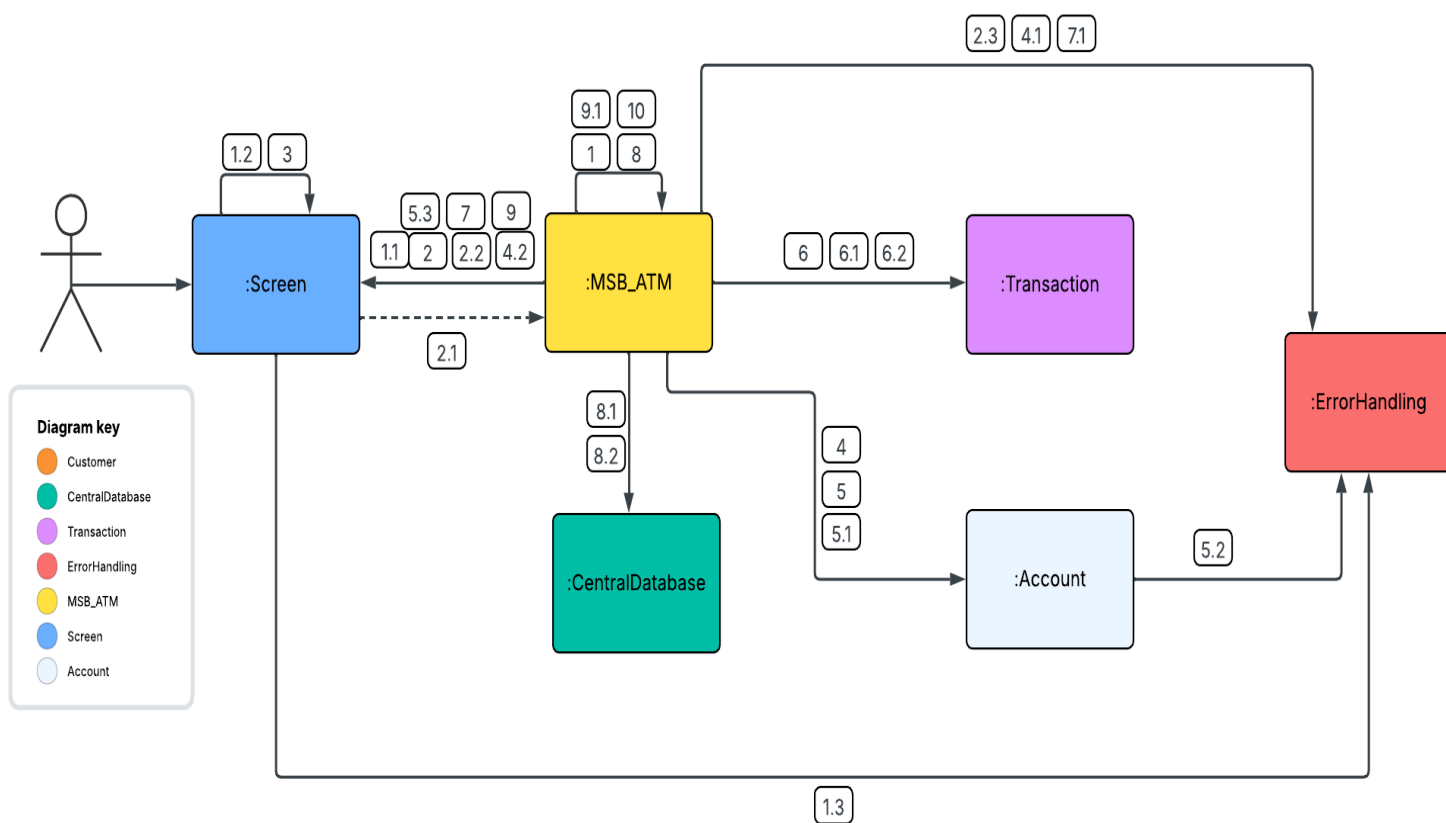
Each Transfer transaction is linked with

- 1 **MSB\_ATM** object
- 1 **Customer** object
- 2 **Account** object (one source account + one destination account)
- 0 or 1 **ConsumerLoanAccount** object (only if overdraft protection is activated)
- 1 **Transaction** object (processing the withdrawal + deposit internally)
- 1 **CentralDatabase** object
- 1 **Screen** object
- 1 **ErrorHandling** object (insufficient funds, invalid account pair)

Author: De-Graft Quansah

Last Update: 12/06/2025

### Communication Diagram



## Operation Sequence

- 1: `select_transfer()`
  - 1.1: `display_source_accounts()`
  - 1.2: `display_service_fee()` (Non-MSB only)
  - 1.3: `terminate_session()` (if fee declined)
- 2: `display_account_types()`
  - 2.1: `return_selected_type()`
  - 2.2: `display_invalid_transfer()` (Invalid account pair)
  - 2.3: `terminate_transaction()`
- 3: `capture_amount()`
- 4: `check_balance()`
  - 4.1: `handle_insufficient_funds()`
  - 4.2: `display_insufficient_funds()`
- 5: `allow_overdraft()` (Preferred only)
  - 5.1: `withdraw_from_loan()`
  - 5.2: `handle_overdraft_limit_reached()`
  - 5.3: `display_transfer_failed()`
- 6: `transfer_funds()`
  - 6.1: `process_transaction()`
  - 6.2: `deposit_funds()`
- 7: `display_transfer_summary()`
  - 7.1: `terminate_transaction()`
- 8: `communicate_DB()`
  - 8.1: `update_balance()`

8.2: `record_transaction()`

9: `display_print_receipt()`

9.1: `print_receipt()`

10: `logout()`

## Scenario Sequence

1 , 2 , 3 , 4 , 6 , 7 , 8 , 9 , 10 – **Successful Transfer (MSB Customer)**

1 , 1.2 , 2 , 3 , 4 , 6 , 7 , 8 , 9 , 10 - **Successful Transfer (Non-MSB Customer Accepts Fee)**

1 , 1.2 , 1.3 - **Non-MSB Customer Declines Fee**

1, 2, 3 , 4 , 4.1 , 4.2 , retry/cancel - **Insufficient Funds (Non-Preferred Customer)**

1, 2, 3 , 4 (insufficient) , 5 , 5.1 , 6 , 7 , 8 , 9 , 10 - **Overdraft Transfer (Preferred Customer)**

1, 2, 3 , 4 , 5 , 5.2 , 5.3 - **Overdraft Limit Reached (Preferred Customer)**

1, 2, 2.1, 2.2, 2.3 - **Invalid Account Pair**

1, 2, 3, 4, 5, 6, 7, 7.1 - **Customer Cancels Before Confirmation**

10 – **System Timeout / Auto Logout**

**Author Name: Dylan Weaver**

**Update: 12/06/2025**

Use Case: **CHECK BALANCE**

**Pre-Conditions:**

- User is authenticated. `[CentralDatabase]:authenticate_user()`
- User session is active and account information is already loaded.  
`[CentralDatabase]:get_user_info()`

**Dialog:**

1. The user selects to **check** their balance from the main menu.
  - `[MSB_ATM]:communicate_DB()`
2. The ATM **displays** the eligible accounts: Checking, Savings, Money Market, Consumer Loan, Mortgage, and Certificates of Deposit (CD).
  - `[Screen]:display_account_types()`
3. The user **selects** one of the accounts to view.
  - `[Account]:select_type(accountType {constraint: one of five types})`
4. The system **retrieves** the balance information and details for each selected account.
  - `[CentralDatabase]:get_account_details()`
5. The ATM verifies and **checks** the available balance.
  - `[Account]:check_balance(amount <= balance)`
6. Once the balances are retrieved from the central database, the ATM **displays** them on screen for the user to view.
  - `[Screen]:display_account_details()`
7. The user is **asked** whether they would like a receipt. `[Screen]:display_print_receipt()`
  - If the user chooses to **print** or view the receipt: `[MSB_ATM]:print_receipt()`
8. If the user **cancels** before viewing or printing the receipt:
  - `[ErrorHandling]:terminate_transaction()`
  - `[MSB_ATM]:logout()`
9. After completion, the ATM **session ends** and returns to the main menu.
  - `[ErrorHandling]:terminate_session()`
  - `[MSB_ATM]:logout()`
  - `[Screen]:display_logon_screen()`

**Post-Conditions:**

- The balance information for the account is **displayed** to the customer.  
`[Screen]:display_account_details()`
- A paper or on screen receipt is **printed**. `[MSB_ATM]:print_receipt()`
- The balance inquiry is **recorded** in the Central Database.  
`[CentralDatabase]:record_transaction()`

**Multiplicity:**

Each balance check is linked with

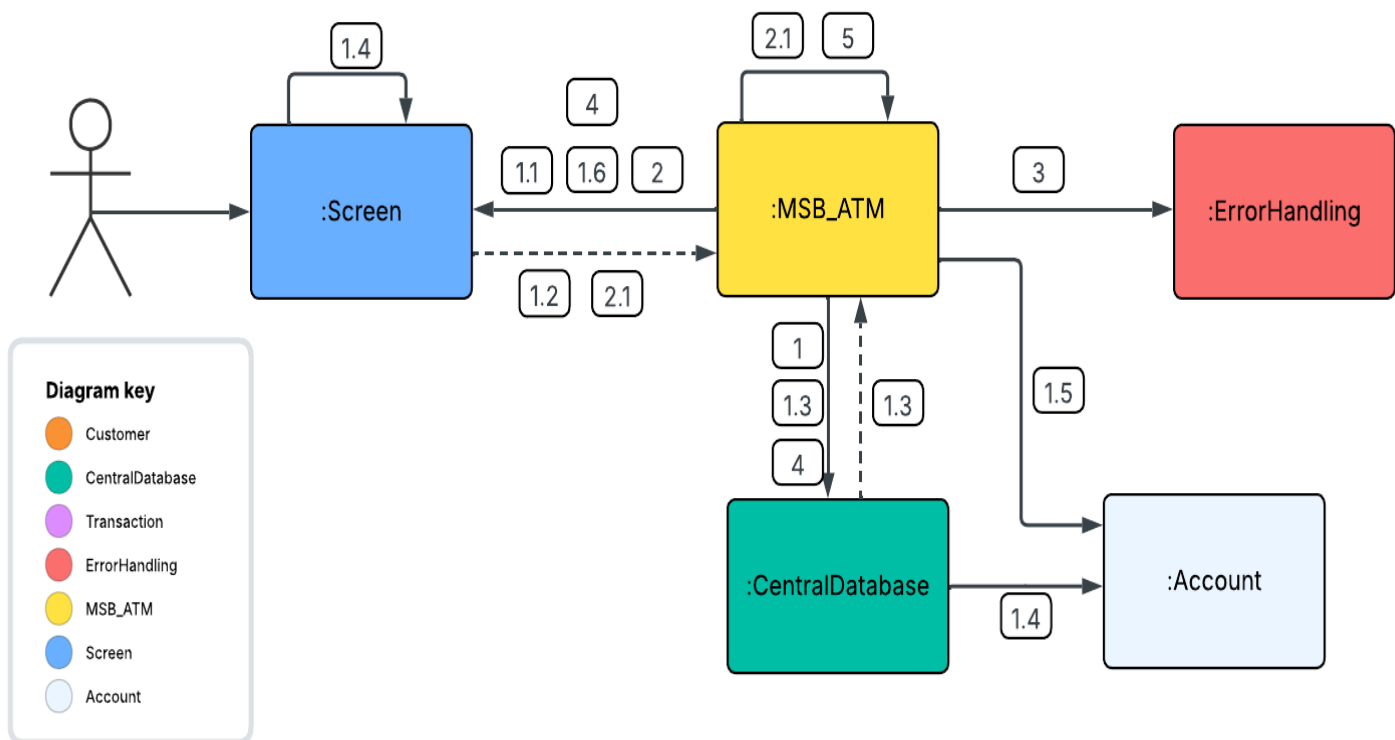
- 1 `MSB_ATM` object
- 1 `Customer` object
- 1 `Account` object (the specific account selected for viewing)
- 1 `Screen` object
- 1 `CentralDatabase` object
- 1 `Transaction` object (logging the balance inquiry)
- 1 `ErrorHandling` object (only if user cancels or makes invalid selection)



**Author: De-Graft Quansah**

**Last Update: 12/06/2025**

### Communication Diagram



### Operation Sequence

- 1: **communicate\_DB()**
- 1.1: **display\_account\_types()**
- 1.2: **return selected\_account(accountType)**
- 1.3: **get\_account\_details()**
- 1.3: **return account\_details()**
- 1.4: **set\_account\_details()**

- 1.5: `check_balance(amount ≤ balance)`
- 1.6: `display_account_details()`
- 2: `display_print_receipt()`
- 2.1: `return confirmation_response(Yes/No)`
- 2.1: `print_receipt()`
- 3: `terminate_transaction()`
- 4: `record_transaction()`
- 5: `logout()`

## Scenario Sequence

1, 1.1, 1.2, 1.3, 1.4, 1.5, 2, 2.1, 4, 5 – **Successful Balance Inquiry (Receipt Printed)**

1, 1.1, 1.2, 1.3, 1.4, 1.5, 1.6, 2, 4, 5 – **User Views Balance But Declines Receipt**

1, 1.1, 1.2, 1.3, 1.4, 1.5, 3, 5 – **User Cancels Before Viewing/Printing Receipt**

5 – **System Timeout / Auto Logout**

**Author: Dylan Weaver**

**Last Update: 10/30/2025**

**Use Case: PRINT RECEIPT**

**Preconditions:**

- User is **authenticated** and has **completed** a transaction (deposit, withdrawal or transfer).  
[CentralDatabase]:authenticate(), [Transaction]:process\_transaction(),  
[CentralDatabase]:record\_transaction()
- User **selects** print receipt when prompted. [Screen]:display\_print\_receipt()

**Dialog:**

1. The ATM **checks** if the machine has paper or if the printer is functional
  - [MSB\_ATM]:check\_printer()
  - If printing is not functional an error message will be **displayed** and the machine will prompt the user for an email to electronically send the receipt.
  - [ErrorHandling]:print\_error()
  - [Screen]:display\_print\_error()
  - [Screen]: capture\_email()
  - [CentralDatabase]:send\_email()
2. The ATM **prints** the receipt if enough paper is available.
  - [MSB\_ATM]:print\_receipt()
3. Once the receipt is **generated** or **displayed**, the ATM system returns to the main menu and ends the session.
  - [MSB\_ATM]:logout()
  - [ErrorHandling]:terminate\_session()
  - [Screen]:display\_logon\_screen()

**Postconditions:**

The receipt is **printed or emailed**. ATM goes back to the logon screen.

**Multiplicity:**

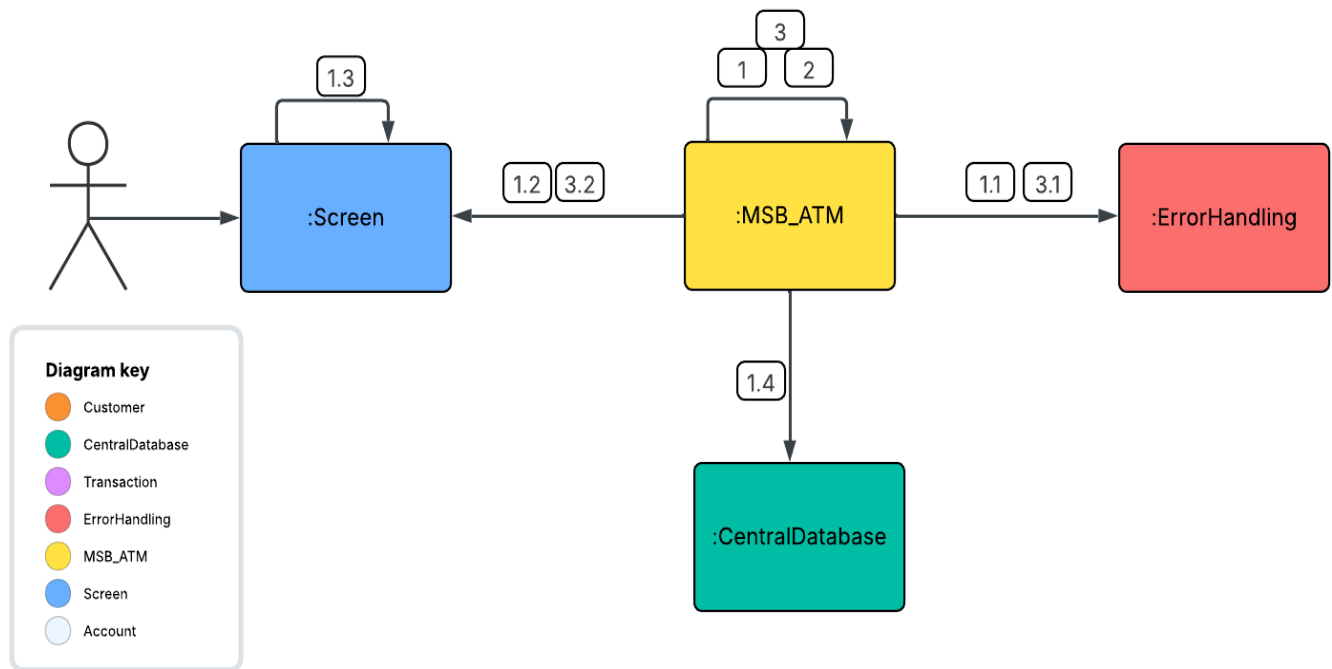
Each Print Receipt use case session is linked with:

- 1 **MSB\_ATM** object
- 1 **CentralDatabase** object
- 1 **Screen** object
- 1 **ErrorHandling** object.

**Author: De-Graft Quansah**

**Last Update: 11/22/2025**

## Communication Diagram



## Operation Sequence

- 1: `check_printer()`
- 1.1: `handle_print_error()`
- 1.2: `display_print_error()`
- 1.3: `capture_email()`
- 1.4: `send_email()`
- 2: `print_receipt()`
- 3: `logout()`

3.1: `terminate_session()`

3.2: `display_logon_screen()`

## Scenario Sequence

1, 2, 3, 3.1, 3.2 – **Successful Print**

1, 1.1, 1.2, 1.3, 1.4, 3, 3.1, 3.2 – **Printer Error (Email Receipt)**

1, 1.1, 1.2, 3, 3.1, 3.2 – **User Cancels at Email Step**

3 – **System Timeout / Auto Logout**

```

// Author Name: Dylan Weaver
// Update: 11/28/2025
// ATM Core Classes (Screen, MSB_ATM, ErrorHandling, CentralDatabase,
// Customer, Account hierarchy, Transaction, Receipt)

//-----
// Includes & basic setup
//-----
#include <string>
#include <vector>

using namespace std;

// Forward declarations for cross-references
class Screen;
class MSB_ATM;
class ErrorHandling;
class CentralDatabase;
class Customer;
class Account;
class CheckingAccount;
class SavingsAccount;
class MoneyMarketAccount;
class ConsumerLoanAccount;
class CDAccount;
class Transaction;
class Receipt;

//-----
// Screen: Handles all on-screen display functions
//-----

```

```

class Screen {
public:
    void display_logon_screen();    // Show welcome/login UI.
    void capture_ID_PW();          // Must capture ID & PIN entered by user.
    void display_system_functions(); // Display main ATM options after login.
    void display_service_menu();    // Display available services menu.
    void display_withdrawal_accounts(); // Show accounts allowed for withdrawal.
    void display_service_fee();     // Inform non-MSB users of $3.50 fee.
    void display_invalid_credentials(); // Show incorrect ID/PIN message.
    void display_account_types();   // Show list of available account types.
    void display_account_details(); // Display details of selected account.
    void display_limit_exceeded();  // Show daily limit exceeded warning.
    void display_insufficient_funds(); // Show insufficient funds message.
    void display_source_accounts(); // Show accounts user can transfer from.
    void display_error();           // General-purpose error display.
    void display_deposit_cancelled(); // Show deposit cancelled status.
    void display_invalid_deposit(); // Show invalid/unreadable deposit error.
    void display_invalid_transfer(); // Show invalid transfer error.
    void display_print_receipt();   // Must ask user Yes/No for receipt.
    void capture_amount();          // Should capture numeric amount from keypad.
    void display_transfer_failed(); // Show transfer failed (insufficient/other).
    void display_transfer_summary(); // Display completed transfer summary.
    void capture_email();           // Should capture email for e-receipt.

private:
    void send_user_selection();     // Must return selected menu option internally.
};

```



```
//-----
// MSB_ATM: Handles all main ATM System functions
//-----

class MSB_ATM {
private:
    string location; // Physical ATM location identifier.
    string managedBy; // Branch or department responsible for this ATM.

    // All operations are private per diagram (called via workflows/controllers).
    void communicate_DB(); // Must initiate communication with CentralDatabase.
    void logout(); // End user session and reset state.
    void complete_transaction(); // Finalize transaction and perform cleanup.
    void capture_deposit(); // Capture inserted bills/checks from hardware.
    void set_withdrawal_amount(); // Read and store withdrawal amount.
    void check_available_funds(); // Internal check before cash dispense.
    void return_funds(); // Return rejected deposit to user.
    void cancel_deposit(); // Abort deposit workflow.
    void validate_deposit(); // Validate deposit content and totals.
    void confirm_deposit(); // Confirm deposit after preview.
    void send_user_response(); // Send Yes/No or menu responses to logic.
    void dispense_cash(); // Physically dispense cash.
    void select_transfer(); // Begin transfer workflow.
    void check_printer(); // Verify printer status (paper/ink).
    void print_receipt(); // Send formatted receipt to printer.

    // Friends allow these collaborating classes to call private methods if needed.
    friend class Transaction;
    friend class ErrorHandling;
    friend class CentralDatabase;
    friend class Screen;
};
```

```
//-----
// ErrorHandling: Handles all errors in the ATM System during a session
//-----

class ErrorHandling {
private:
    void handle_invalid_credentials();    // Called after wrong PIN/ID attempts.
    void handle_invalid_deposit();        // Called when deposit is invalid.
    void cancel_transaction();            // Must safely abort current transaction.
    void terminate_session();             // Ends session and returns to login.
    void terminate_transaction();         // Internal cleanup for failed transaction.
    void handle_overdraft_limit_reached(); // Overdraft/loan exceeds allowed limit.
    void handle_insufficient_funds();     // Not enough available funds.
    void handle_print_error();            // Printer malfunction or no paper.

    friend class Transaction;
    friend class Account;
    friend class MSB_ATM;
    friend class Screen;
};

//-----
// CentralDatabase: Handles all Database related operations
//-----

class CentralDatabase {
private:
    vector<Customer> customerInfo;    // All customer account profiles.
    vector<string> transactionLog;    // History of all ATM transactions.

    void authenticate_user(int id, int pw, bool& status);
        // Validate ID & PIN against stored credentials.

    void get_user_info();

```

```

        // Load user profile information for current session.

void get_account_details();
    // Load account details (balances, limits, types).

void update_balance();
    // Commit updated balances for one or more accounts.

void record_transaction();
    // Append new transaction entry to transactionLog.

void store_customer_info();
    // Persist any changes to customer profile data.

void interact_with_banks();
    // Handle interactions with external/non-MSB banks.

void send_email();
    // Send email receipts or notifications.

friend class Transaction;
friend class MSB_ATM;
friend class Customer;
};
//-----
// Customer: Handles Customer related information
//-----
class Customer {
private:
    string name;        // Customer full name.
    string address;     // Mailing/physical address.

```

```

int  userID;      // Unique login/ID value.
int  PIN;         // PIN associated with card.
bool preferredStatus; // Preferred-customer flag (overdraft & hold rules).

void set_user_info();
    // Must populate all customer fields from CentralDatabase.

void set_account_balance();
    // Set current balances for all customer accounts in session.

friend class CentralDatabase;
friend class Transaction;
};
//-----
// Account: Base class for all account types
//-----
class Account {
private:
    int  accountNumber; // Unique account identifier.
    string accountType; // Checking, Savings, MM, Loan, Mortgage, etc.
    double accountBalance; // Current balance.
    int  numHoldDays; // Hold period (e.g., 3 days for some deposits).

    void set_account_details();
        // Must set base account fields when account is selected.

    bool check_balance(double amount);
        // Returns true if amount can be covered under current rules.

    void allow_overdraft(bool preferredStatus, double overdraftAmount);
        // Apply overdraft rules for preferred customers.

```

```

void withdraw_from_loan(double amount, double limit);
    // Base loan withdrawal helper (overridden/refined by subclasses).

void apply_hold(bool isPreferredCustomer, bool check);
    // Non-preferred + check deposits: apply required hold days.

friend class Transaction;
friend class CheckingAccount;
friend class ConsumerLoanAccount;
friend class CDAccount;
};
//-----
// CheckingAccount: Subclass of Account, handles specific Checking behavior
//-----
class CheckingAccount : public Account {
private:
    void allow_overdraft();
        // Should invoke base overdraft logic with checking-specific rules.

    friend class Transaction;
};
//-----
// CDAccount: Subclass of Account, models Certificate of Deposit (read-only)
//-----
class CDAccount : public Account {
private:
    void check_balance();
        // CDs are read-only in ATM: only balance inquiry allowed.

    friend class Transaction;

```

```

};
//-----
// ConsumerLoanAccount: Subclass of Account, handles loan withdrawals
//-----
class ConsumerLoanAccount : public Account {
private:
    void withdraw_from_loan(double amount, double limit);
        // Must ensure withdrawal does not exceed the remaining loan limit.

    friend class Transaction;
};
//-----
// Transaction: Handles Deposits, Withdrawals, Transfers
//-----
class Transaction {
protected:    // From diagram: all fields have '#'
    int    transactionID; // Auto-generated transaction identifier.
    double amount;        // Current transaction amount.
    string timeStamp;      // Time the transaction occurred.
    string transactionType; // deposit / withdraw / transfer / inquiry.
    double serviceFee;     // Fee for non-MSB customers.

private:    // From diagram: all operations use '-'

    // Collaborating objects used in the sample implementation.
    Screen    screen;      // For prompts and input capture.
    ErrorHandling    error; // For reporting and handling errors.
    CentralDatabase    centralDB; // For persistence of balances/logs.
    MSB_ATM    msbATM;     // For physical ATM actions (cash, deposit).
    Account    account;     // Generic account used in simple flows.
    Account    fromAccount; // Source account in transfer.

```

```

double        dailyWithdrawn;// Tracks total withdrawal for the day.

void withdraw_funds(double amount);
    // Must check balance, enforce daily limit, then reduce balance.

void check_limit(double withdrawn, double limit);
    // Should ensure total withdrawn today does not exceed limit.

void deposit_funds(double amount, const string& accountType);
    // Must add deposit amount and apply hold rules when needed.

void transfer_funds();
    // Must move the amount from source to destination account.

void validate_amount();
    // Check for non-negative, non-zero, and rule-compliant amounts.

void process_transaction();
    // High-level wrapper: validate, update DB, and finalize.

friend class MSB_ATM;
};
//-----
// Receipt: Represents printing or emailing a receipt
//-----
class Receipt {
public:
    void print_receipt();    // Send formatted receipt to printer.
    void send_email_copy(); // Send same receipt via email.
};
//-----

```

```

// Sample Implementation for Transaction operations
//-----
// Withdraw funds from an account – follows diagram + operation sequence.
void Transaction::withdraw_funds(double amount) {
    screen.capture_amount();
    // Should capture the withdrawal amount from the user keypad.

    validate_amount();
    // Ensure amount is positive, non-zero, and respects basic rules.

    if (!account.check_balance(amount)) {
        // Must verify sufficient available funds (non-preferred = no overdraft).
        error.handle_insufficient_funds();
        // Show error message for low balance and prompt for next action.
        return;
    }

    check_limit(dailyWithdrawn, 500.0);
    // Must enforce the $500 daily withdrawal limit.

    centralDB.update_balance();
    // Must update stored account balance in CentralDatabase
    // (internal logic subtracts 'amount' from the correct account).

    msbATM.dispense_cash();
    // Physically dispense cash after balance update succeeds.

    record_transaction();
    // Must log this withdrawal into the transaction history.
}
// Deposit funds into an account – follows deposit operation sequence.

```



```

void Transaction::deposit_funds(double amount, const string& accountType) {
    screen.display_account_details();
    // Must show the selected account details before deposit begins.
    msbATM.capture_deposit();
    // Should scan/collect inserted bills or checks from deposit hardware.
    // For non-preferred customers, check deposits require a 3-day hold.
    // Preferred customers bypass this hold.
    apply_hold(/*isPreferredCustomer=*/false, /*check=*/true);

    centralDB.update_balance();
    // Must add the deposit amount to the selected account in DB.

    record_transaction();
    // Must log the deposit with timestamp, type, and amount.
}
// Transfer funds between accounts – follows transfer operation sequence.
void Transaction::transfer_funds() {
    screen.display_source_accounts();
    // Must display available source accounts to transfer FROM.

    screen.capture_amount();
    // Must capture the transfer amount from the user keypad.

    validate_amount();
    // Ensure transfer amount is valid before processing.

    if (!fromAccount.check_balance(amount)) {
        // Must ensure enough funds in source account or overdraft allowed.
        error.handle_insufficient_funds();
        // Show insufficient funds error and abort transfer.
        return;
    }
}

```

```
}

process_transaction();
    // Wrapper: will apply overdraft rules, update both accounts, etc.

centralDB.update_balance();
    // Must subtract the transfer amount from the source account.

centralDB.update_balance();
    // Must add the same amount to the destination account
    // (internally selecting the correct destination account).
log_transaction();
    // Should record full transfer details (source, destination, amount, time).
}
```

# Glossary

---

## A

### **Account**

**A financial entity representing a customer's Checking, Savings, Money Market, Consumer Loan, Mortgage, or CD account. Stores balance, type, and related state used in ATM transactions.**

### **Account Balance**

**The current monetary value stored in an account. Used for withdrawals, transfers, deposits, and balance inquiries.**

### **Account Details**

**Information retrieved from the Central Database for a selected account, including type, balance, loan limit (if applicable), and hold days.**

### **Account Type**

**A classification that determines which operations are allowed (e.g., withdrawals, deposits, transfers). Options include Checking, Savings, Money Market, Consumer Loan, Mortgage, and CD.**

### **Authentication**

**The process in which the ATM verifies a user's ID and PIN against the Central Database before allowing any transactions.**

### **Auto Logout (Timeout)**

**A system-triggered logout that occurs after a period of inactivity to protect customer security.**

---

**B****Balance Inquiry**

A read-only transaction that allows the user to view the balance of any account, including CDs.

**Bank Interaction (External)**

Function performed by the CentralDatabase to communicate with other banks for non-MSB customer transactions.

---

**C****Capture Amount**

Input operation where the ATM collects a numerical transaction amount from the user.

**Capture Deposit**

Hardware-level process where the ATM reads inserted checks or cash.

**CDAccount**

A subclass of Account for Certificates of Deposit, allowing only balance inquiries.

**Central Database**

The main storage system holding customer data, account balances, loan limits, daily totals, and transaction histories. All ATMs communicate with this database.

**Check Balance (Operation)**

Verifies whether sufficient funds exist in an account or whether overdraft rules apply.

**Check Deposit (Validation)**

Process where the ATM inspects a check to ensure it is readable and valid.

**CheckingAccount**

A subclass of Account supporting withdrawals, deposits, and overdraft protection for preferred customers.

**Communication Diagram**

A UML diagram showing how system objects collaborate to complete a specific use case.

**ConsumerLoanAccount**

A subclass of Account handling loan withdrawals and overdraft transfers.

**Customer**

Represents a user with accounts, credentials, preferred status, and associated banking privileges.

**Customer Experience**

Narrative describing team workflow, challenges, and collaboration during the project.

---

**D****Daily Withdrawal Limit**

A restriction limiting customers to \$500 per day across all withdrawals.

**Deposit Hold**

A bank rule delaying availability of check deposits for non-preferred customers.

---

**E****Email Receipt**

A receipt delivered electronically when the ATM printer fails or upon user request.

**Error Handling**

Logic responsible for responding to invalid input, insufficient funds, overdraft failure, or hardware problems.

---

**F****Fee (Non-MSB Customers)**

A \$3.50 charge applied per transaction when the customer does not belong to Montana Savings Bank.

**Flow of Events (Dialog)**

Step-by-step description of user interactions and system responses in a use case.

---

**H****Hardware Constraints**

Limitations of ATM physical components—e.g., cash supply, printer paper, deposit reader.

---

**I****Implementation**

C++ source code that realizes the system's operations, classes, and workflows.

**Index**

Alphabetical list of important terms and concepts in the final project document.

**Insufficient Funds**

An error state where withdrawal or transfer amount exceeds the allowable balance or overdraft limit.

**Invalid Credentials**

**Authentication failure due to mismatched ID or PIN.**

**Invalid Deposit**

**Deposit flagged as unreadable, empty, or improperly formatted.**

**Invalid Transfer**

**Transfer attempt violating MSB rules (e.g., from Consumer Loan to Checking).**

---

**L****Loan Withdrawal**

**Process of withdrawing money from a Consumer Loan account, constrained by the loan limit.**

**Logon Use Case**

**Use case describing how users authenticate and access ATM services.**

**Logout**

**Action that ends a session and resets the ATM to the welcome screen.**

---

**M****Make Deposit Use Case**

**Use case describing deposit workflows, including validation and hold rules.**

**Make Transfer Use Case**

**Use case describing inter-account fund transfers and overdraft handling.**

**Make Withdrawal Use Case**

**Use case describing withdrawal logic, including limits, overdraft, and physical cash dispensing.**

**Mortgage Account**

**Customer loan account representing mortgage balances. Supports balance inquiry and loan payment.**

---

**O****Object-Oriented Design (OOD)**

**Modeling approach using UML diagrams to represent classes, objects, operations, and interactions.**

**Operation Sequence**

**Ordered list of communication steps between system objects for a use case.**

**Overdraft Protection**

**Feature allowing preferred customers to overdraft from their Consumer Loan account.**

---

**P****Preferred Customer**

**Customer with special privileges: no deposit holds and overdraft support.**

**Print Receipt Use Case**

**Use case handling receipt printing, printer errors, and email backup.**

**Printer Error**

**Condition where paper or hardware malfunction prevents printing.**

---

**R**



**Receipt**

**Document summarizing a transaction, either printed or emailed.**

**Record Transaction**

**Database operation logging each ATM transaction.**

**Requirements Specification**

**Formal description of functional and non-functional features the ATM system must support.**

---

**S****SavingsAccount**

**Account type supporting deposits, withdrawals, transfers, and interest rules.**

**Scenario Sequence**

**List summarizing paths the user may take during a use case.**

**Screen Class**

**Class responsible for all user-interface displays and input prompts.**

**Service Fee**

**A \$3.50 fee for non-MSB customers.**

**Software Engineering Process**

**The structured phases of requirements, design, implementation, and testing used in the project.**

**Static Relationship Diagram**

**UML diagram showing permanent associations among classes.**

**System Timeout**

**Inactivity-triggered automatic logout.**

---

## **T**

### **Transaction Class**

**Class responsible for deposits, withdrawals, transfers, and transaction logging.**

### **Transaction History**

**List of recorded transactions stored in the central database.**

### **Transfer Funds Use Case**

**Use case describing how users move funds between accounts.**

---

## **U**

### **UML**

**Unified Modeling Language — used for diagrams representing system structure and behavior.**

### **Update Balance**

**Database operation changing the stored balance of an account.**

### **Use Case Descriptions**

**Detailed specifications outlining each system interaction.**

---

## **W**

### **Withdrawal**

**Transaction where cash is dispensed to the user.**

### **Withdraw Funds Use Case**

**Use case defining withdrawal operations and rules.**

# PROJECT INDEX

---

## A

|                                        |                                      |
|----------------------------------------|--------------------------------------|
| <b>Account Class .....</b>             | <b>13–14, 21, 52–60</b>              |
| <b>Account Balance .....</b>           | <b>11, 23–24</b>                     |
| <b>Account Details .....</b>           | <b>20, 23–24</b>                     |
| <b>Account Types .....</b>             | <b>6, 20, 23</b>                     |
| <b>Authentication .....</b>            | <b>6, 10, 23–24, 27, 30, 44, 52</b>  |
| <b>ATM (MSB_ATM Class) .....</b>       | <b>6–7, 18–20, 23–60, 31, 35, 41</b> |
| <b>ATM Location / Management .....</b> | <b>18–19</b>                         |
| <b>Auto Logout (Timeout) .....</b>     | <b>26, 29, 32, 37, 43, 47, 51</b>    |

---

## B

|                                              |                         |
|----------------------------------------------|-------------------------|
| <b>Balance Inquiry .....</b>                 | <b>11, 44–47, 52–60</b> |
| <b>Bank Interaction (External) .....</b>     | <b>15</b>               |
| <b>Branch Database Synchronization .....</b> | <b>6–7</b>              |
| <b>Business Rules .....</b>                  | <b>10–11, 27–29</b>     |

---

## C

|                               |                              |
|-------------------------------|------------------------------|
| <b>Capture Amount .....</b>   | <b>20, 27, 33, 38</b>        |
| <b>Capture Deposit .....</b>  | <b>18, 28–29, 33–37</b>      |
| <b>CDAccount Class .....</b>  | <b>14, 21, 52–60</b>         |
| <b>Central Database .....</b> | <b>6–7, 10–11, 15, 23–60</b> |

|                                         |                           |
|-----------------------------------------|---------------------------|
| <b>Check Balance (Operation)</b> .....  | 11, 44–47, 52–60          |
| <b>Check Deposit (Validation)</b> ..... | 10, 33–37                 |
| <b>CheckingAccount Class</b> .....      | 16, 52–60                 |
| <b>Communication Diagrams</b> .....     | 25, 30–31, 35, 41, 46, 50 |
| <b>ConsumerLoanAccount Class</b> .....  | 16, 52–60                 |
| <b>Customer Class</b> .....             | 6–7, 17, 23–60, 38, 44    |
| <b>Customer Experience (Team)</b> ..... | 8–9                       |

---

## D

|                                             |                  |
|---------------------------------------------|------------------|
| <b>Daily Limit (\$500)</b> .....            | 10, 27–29, 31–32 |
| <b>Deposit (Use Case)</b> .....             | 10–11, 33–37     |
| <b>Deposit Holds (3-Day Hold)</b> .....     | 10–11, 33        |
| <b>Deposit Options</b> .....                | 10–11            |
| <b>Display Screens (Screen Class)</b> ..... | 20               |
| <b>Dynamic Behavior (Sequence)</b> .....    | 26–51            |

---

## E

|                                                    |                          |
|----------------------------------------------------|--------------------------|
| <b>Email Receipt</b> .....                         | 14, 20, 28–29, 38, 48–51 |
| <b>ErrorHandling Class</b> .....                   | 18, 52–60                |
| <b>Errors (Deposit, Transfer, Overdraft)</b> ..... | 18, 27–29, 33–37, 41–43  |

---

## F

|                                       |                       |
|---------------------------------------|-----------------------|
| <b>Fee (Non-MSB Customers)</b> .....  | 6–7, 10–11, 23–37, 38 |
| <b>Flow of Events (Dialogs)</b> ..... | 23–51                 |

**Funds Transfer ..... 11, 38–43, 52**

---

## **G**

**Glossary ..... End of Document**

**Group Work Experience ..... 8–9**

---

## **H**

**Hold (Deposit Hold Rules) ..... 10–11, 33–37**

**Hardware Constraints (ATM) ..... 18–20**

---

## **I**

**Implementation (C++ Code) ..... 52–60**

**Index ..... End of Document**

**Insufficient Funds ..... 10, 18, 27–29, 33–37, 41–43**

**Invalid Credentials ..... 23–24, 30–31**

**Invalid Deposit ..... 33–37, 52–60**

**Invalid Transfer ..... 38–43, 52–60**

---

## **L**

**Loan Withdrawal (Consumer Loan) ..... 6–7, 10–11, 16, 27–29**

**Logon Use Case ..... 23–26, 52–60**

|                                      |                     |
|--------------------------------------|---------------------|
| <b>Logout (Session End) .....</b>    | <b>26, 51</b>       |
| <b>Loan Limits (Overdraft) .....</b> | <b>10–11, 27–29</b> |

---

## **M**

|                                       |                          |
|---------------------------------------|--------------------------|
| <b>Make Deposit Use Case .....</b>    | <b>33–37</b>             |
| <b>Make Transfer Use Case .....</b>   | <b>38–43</b>             |
| <b>Make Withdrawal Use Case .....</b> | <b>27–32</b>             |
| <b>Mortgage Account .....</b>         | <b>6–7, 10–11, 38–43</b> |

---

## **O**

|                                               |                                  |
|-----------------------------------------------|----------------------------------|
| <b>Object-Oriented Design (OOD) .....</b>     | <b>6–7</b>                       |
| <b>Operation Sequences .....</b>              | <b>26, 31, 36, 42–43, 47, 51</b> |
| <b>Overdraft (Preferred Customer Rule) ..</b> | <b>6–7, 10–11, 16, 27–29</b>     |

---

## **P**

|                                              |                     |
|----------------------------------------------|---------------------|
| <b>Page Numbers (Document Mapping) .....</b> | <b>1–63</b>         |
| <b>Preferred Customer Status .....</b>       | <b>6–7, 10–11</b>   |
| <b>Print Receipt Use Case .....</b>          | <b>48–51</b>        |
| <b>Printer Errors .....</b>                  | <b>48–51, 18–20</b> |

---

## R

|                                         |                             |
|-----------------------------------------|-----------------------------|
| <b>Receipt (Print or Email) .....</b>   | <b>11, 20, 28–29, 48–51</b> |
| <b>Record Transaction .....</b>         | <b>15, 27–37, 41–51</b>     |
| <b>Requirements Specification .....</b> | <b>6–12</b>                 |

---

## S

|                                           |                               |
|-------------------------------------------|-------------------------------|
| <b>SavingsAccount Class .....</b>         | <b>16, 52–60</b>              |
| <b>Scenario Sequences .....</b>           | <b>26, 32, 37, 43, 47, 51</b> |
| <b>Screen Class .....</b>                 | <b>20, 52–60</b>              |
| <b>Service Fee (\$3.50) .....</b>         | <b>6–7, 10–11, 23–37, 38</b>  |
| <b>Software Engineering Process .....</b> | <b>6–9</b>                    |
| <b>Static Relationship Diagram .....</b>  | <b>22</b>                     |
| <b>System Overview .....</b>              | <b>6–7</b>                    |
| <b>System Timeout .....</b>               | <b>26–51</b>                  |

---

## T

|                                      |                                |
|--------------------------------------|--------------------------------|
| <b>Team Information .....</b>        | <b>8–9</b>                     |
| <b>Transaction Class .....</b>       | <b>21, 27–37, 41–51, 52–60</b> |
| <b>Transaction History .....</b>     | <b>6–7, 10–11, 15, 28–37</b>   |
| <b>Transfer Rules (MSB) .....</b>    | <b>10–11, 38–43</b>            |
| <b>Transfer Funds Use Case .....</b> | <b>38–43</b>                   |

---

## U

**UML (Use Case, Class, Communication) ..... 12–22**

**Update Balance ..... 15, 27–37, 41–51**

**Use Case Descriptions ..... 23–51**

---

## W

**Withdrawal (Daily Limit, \$10 multiples) ..... 10, 27–32**

**Withdraw Funds Use Case ..... 27–32**

---